[Experiment, Analysis, and Benchmark] Filtered Vector Search Under Updates: When Does an Index Beat a Scan?

Suan Lee
Semyung University, Jecheon, Republic of Korea
suanlee@semyung.ac.kr ORCID: https://orcid.org/0000-0002-3047-1167

***Abstract*—Vector databases must answer nearest-neighbor queries with attribute filters over data that is continuously inserted and deleted, yet filtered search has been evaluated almost exclusively on static data. We present an experimental study of filtered vector search under updates. It covers nine methods from seven families: exact pre-filter scans, post-filtering, a predicate-agnostic in-filter graph (ACORN), a label-specialized graph (Filtered-DiskANN), a label-indexed tree (Curator), a partitioned index with a bitmap filter (IVF), and a regenerable pool index. The workloads use SIFT, DEEP (at three scales), and YFCC collections of up to 10^7 vectors, label predicates with controlled selectivity and attribute–geometry correlation, real multi-tag predicates, and three update streams, one of which concentrates deletions on the attribute. First, at recall 0.95 and above, an exact scan of label-partitioned admissible points beats every index up to 10^4 admissible points, and at a fixed admissible-set size the indexes' advantage shrinks as the collection grows. Second, on real predicates no index reaches recall 0.99 without a selectivity dispatcher; with one tuned on the queries, a bitmap-filtered IVF index is the fastest of the scan-based choices and pairing a per-label index (Curator) with IVF is faster still, although evaluating the predicates themselves costs about as much as the search. Third, under churn, IVF's exact deletions keep it accurate at two to three orders of magnitude less maintenance than the graph indexes. ACORN's maximum recall falls from 1.000 to 0.575 when deletions correlate with the attribute—an effect of its update path, since a fresh build on the same live data reaches 0.995— and at recall 0.95 a freshly rebuilt Filtered-DiskANN pays off only when queries outnumber updates by a stream- and round-dependent factor of at least 1.3; on one stream it never does at recall 0.99. Fourth, rankings change with the measurement setup: with 28 threads ACORN or post-filtering overtakes IVF on some static workloads, and shared-machine contention, a scalar default build of ACORN, and two other implementation defects each distorted measurements before we controlled for them; contention alone reversed which method wins at one admissible-set size. We describe the protocol and the fixes, and release all artifacts.**

***Index Terms*—vector search, approximate nearest neighbor search, filtered search, hybrid queries, dynamic indexing, benchmark**

I. Introduction

Vector databases answer nearest-neighbor queries that carry attribute predicates—"similar products *in stock*", "passages *from this tenant*"—over collections that change continuously. A large body of recent work builds indexes for one of these two requirements: filter-aware graphs that keep navigation inside a predicate [2], [3], [33], [32], range-filter structures [28], [29], [30], and streaming indexes that absorb insertions and deletions [6], [17], [18]. Experimental studies have compared filtered methods on static data [7], [8], [9], [10], [11], and benchmarks have exercised streaming updates without filters [5], [16]. A deployment, however, faces both at once, and the experimental study of Li et al. lists dynamic updates as an open problem for filtered search [8]. This paper evaluates filtered search *under updates*, against simple baselines—an exact scan, a bitmap-filtered IVF index, and a selectivity dispatcher—that are easy to deploy alongside any index.

We ask six questions. **Q1**: When does an index beat an exact scan of the admissible points, and is the crossover a function of the admissible-set size $s \cdot n$? **Q2**: How does the correlation between attributes and geometry reorder the methods? **Q3**: What happens on real predicates whose selectivity spans orders of magnitude? **Q4**: How do accuracy, throughput, footprint, and maintenance evolve as the data churns, and what if deletions correlate with the attribute? **Q5**: Which method is cheapest once maintenance and queries are both paid for? **Q6**: Which evaluation practices change the conclusions?

Summary of findings. (1) At recall 0.95 and above, an exact scan of label-partitioned admissible points wins whenever they number 10^4 or fewer, and at a *fixed* admissible-set size the indexes' advantage shrinks as the collection grows: with 10^5 admissible points, IVF is $3.6\times$ faster than the scan at recall 0.95 when $n{=}10^6$ but only $1.0\times$ when $n{=}10^7$, plausibly because an index's cost grows with the space it navigates while the scan's does not; Curator, which indexes each label separately, is the exception: its ratio to the scan stays nearly constant at 10^4 admissible points and falls far less than IVF's at 10^5. (2) Correlation reorders the methods: post-filtering is among the fastest when attributes are independent of geometry, and its maximum recall falls below 0.95 on SIFT1M when they are correlated; a label-specialized graph behaves the opposite way. (3) On real tag predicates (YFCC-10M) no index reaches recall 0.99 by itself; with a selectivity dispatcher that scans small admissible sets and is tuned on the queries, every index does, plain IVF with a bitmap filter is the fastest scan-based dispatch, and pairing a per-label index for selective predicates (Curator) with IVF or a graph for common ones is faster still (applied unchanged to 500 disjoint queries, the tuned dispatchers reach 0.984–0.994 and

keep their order); evaluating the tag predicates costs about as much as the IVF search itself. (4) Under churn, IVF's exact deletions keep it flat at a maintenance cost two to three orders of magnitude below the graph indexes' (119–888× less); ACORN, whose only deletion mechanism is the predicate bitmap, drifts under uniform deletions and collapses (maximum recall 1.000→0.575) when deletions correlate with the attribute (a fresh build on the same live data reaches 0.995); Filtered-DiskANN must be rebuilt to stay fresh, at 659× IVF's maintenance on DEEP-5M. (5) At recall 0.95, a freshly rebuilt Filtered-DiskANN beats IVF in total cost only above 1.3–2.2 queries per update on DEEP-5M and 17.8–38.4 on SIFT-500K, depending on the round (wall time; in 2 of 6 SIFT-500K rounds not at all), and at recall 0.99 it never does on SIFT-500K; under correlated deletions, which slow IVF, it pays off at 0.9–2.1 queries per update at recall 0.99; with single-threaded queries every other index costs more than IVF on both terms, hence at every update-to-query ratio. (6) Rankings depend on the measurement setup: ACORN and post-filtering scale better with threads than IVF and overtake it at 28 threads on some static workloads; machine contention changed throughputs by up to 2.9× between otherwise identical runs; and ACORN's documented build compiles scalar distance kernels, while two baselines have defects on paths that benchmarks rarely exercise.

Contributions. A benchmark for filtered search under updates: workload generators with controlled selectivity and correlation, real multi-tag predicates, three update streams including a correlated-deletion stream, and a measurement protocol that gates every run on a calibration kernel (Section III); a study of nine methods from seven families (Sections IV–VI); a cost model for choosing between rebuilding and incremental maintenance (Section VI); lessons for evaluation, including a fix for a defect in ACORN's incremental insertion (Section VII); and all artifacts, including per-query results.

II. Filtered Search and Its Update Paths

A filtered k-NN query asks for the k points nearest to q among the *admissible set* $A_p = \{x \in X : p(x)\}$ of a predicate p; its selectivity is $s = |A_p|/n$. We group the methods we evaluate into seven families by *where the predicate enters the search* and record, for each, *how it absorbs insertions and deletions* (Table I).

Pre-filter scan. Compute A_p, then scan it exactly. When the predicate is an equality on a label known at insertion time, the points of each label can be stored contiguously (*per-label partition*), so the scan touches only $|A_p|$ vectors; we also evaluate the general case in which A_p is an arbitrary bitmap and the scan visits all n rows and skips inadmissible ones (*bitmap scan*). Updates are appends and removals; there is nothing to repair.

Post-filter. Search an unfiltered index for $k' \geq k$ candidates and keep the admissible ones. We use hnswlib [1] with slot reuse for deletions. Post-filtering needs $k' \gtrsim k/s$ when admissible points are spread like the data, and fails when they are not.

TABLE I
Method families, how each consumes the predicate, and how each absorbs updates in the implementations we evaluate.

family	method (impl.)	predicate	insert	delete
pre-filter scan	per-label / bitmap (Faiss)	any†	append	remove
post-filter	HNSW (hnswlib)	any	add	slot reuse
in-filter graph	ACORN-γ, ACORN-1	any	add	bitmap exclusion
label graph	Filtered-DiskANN (diskannpy)	eq. label	rebuild	rebuild
label tree	Curator	label‡	add+grant	revoke label
partition+bitmap	IVF-Flat (Faiss)	any	append	remove
regenerable pools	RIG (ours)	any	route+append	remove

†The per-label partition requires an equality predicate on a build-time label; the bitmap scan accepts any. ‡Other predicates through a bitmap-filtered search.

Predicate-agnostic in-filter graph. ACORN-γ [2] builds an HNSW-like graph whose neighbor lists are expanded by a factor γ, so that the subgraph induced by an arbitrary predicate stays navigable; search visits only admissible nodes. Insertions use the graph's incremental `add`; ACORN has no deletion routine, so deleted points are excluded through the predicate bitmap and remain in the graph as routers. We also evaluate ACORN-1 (γ=1), its cheaper-to-build variant.

Label-specialized graph. Filtered-DiskANN [3] builds a Vamana graph whose pruning preserves, for every label, paths among the points carrying it, and starts each query from a per-label medoid. It supports equality on build-time labels only. The public Python binding exposes no dynamic filtered index, so freshness requires rebuilding.

Label-indexed tree. Curator [26] builds one hierarchical clustering tree over all vectors and embeds a lightweight per-label index in it as a subtree; a point is indexed under a label by granting it the label, and a label search traverses that label's subtree. Other predicates, such as the AND of two tags, use its bitmap-filtered search. Insertion adds the vector to the tree and grants its labels. The released implementation rejects vector removal, so deletion revokes the point's labels: the vector stays in the tree, but no label search can return it.

Partition with bitmap. IVF-Flat [4] assigns points to the nearest of n_{list} k-means centroids and scans the n_{probe} lists nearest to the query, skipping inadmissible entries via a bitmap. With centroids fixed after training, insertion is an append to one list and deletion an exact removal; there are no tombstones and no graph to repair. Correlated predicates hurt IVF in a specific way: the admissible points may live in lists far from the query, so recall at a fixed n_{probe} drops.

Regenerable pool index. RIG is a design point of our own that stores no adjacency lists: points belong to overlapping pools produced by hash-priority ball carving, and the neighbors of a point are recomputed at query time from its pools, inside the predicate (filtered navigation with predicate-first scans). Its state is a function of the point set, which gives exact deletions and history-independent maintenance; we include it because it occupies the corner of the design space between IVF (partitions, exact updates) and graphs (navigation), not as the subject of the study. Its dynamic path routes insertions down a fixed leader tree and compacts every round.

III. Benchmark Design

Datasets and predicates. We use SIFT1M (n=10^6, d=128), DEEP at 10^6, 3.16×10^6, and 10^7 points (d=96, normalized; the two smaller sets are fixed random subsets of DEEP-10M, so that n varies without changing the distribution), and YFCC-10M from the filtered track of the NeurIPS'23 Big-ANN competition [5] (n=10^7, d=192), whose points carry real tag sets and whose queries ask for the AND of one or two tags. On SIFT and DEEP we generate label predicates with two controlled knobs. Each point receives one of $L = 1/s$ labels, defined by quantiles of $\beta\, z(x) + (1-\beta)\,\varepsilon$, where $z(x)$ is the standardized projection $\langle x, w\rangle$ on a fixed random direction w and ε is standard Gaussian noise, so that s sets the selectivity and $\beta \in [0,1]$ controls the correlation between label and geometry (β=0: independent labels; β=1: slabs along w); each query asks for one uniformly random label. We use $s \in \{0.5, 0.1, 0.01\}$ and $\beta \in \{0, 0.5, 0.9, 1\}$. On YFCC we use 500 fixed random public queries; their selectivities range from 7.4×10^{-6} (74 admissible points) to 0.19, with 10th, 50th, and 90th percentiles of 2.2×10^{-5}, 1.3×10^{-3}, and 8.1×10^{-2}, and our exact filtered ground truth matches the official one on all of them.

Update streams. Three streams combine filtered queries (β=0.9, s=0.1) with churn. *SIFT-500K*: SIFT descriptors drawn from BIGANN (1.5×10^6 distinct vectors, 500 BIGANN queries), 5×10^5 live points, 20 rounds, each deleting 10% of the live points uniformly at random and inserting 5×10^4 unseen points (200% turnover). *DEEP-5M*: 5×10^6 live points, 10 rounds of 10% deletions and 4.99×10^5 insertions of unseen DEEP vectors (100% turnover). A third stream, *SIFT-500K-adv*, tests correlation $\times$ churn: it is SIFT-500K with the same insertions and queries, but each round deletes the 10% of live points with the largest projection on the label direction w (plus small noise), so deletions empty the geometric regions of the highest labels first and the admissible sets of those labels shrink while others grow. Exact filtered ground truth is recomputed over the live set after every round. As a control for the update paths, we also snapshot the live set of both SIFT streams after round 20 and build every method from scratch on it, with the same queries and ground truth. Every method applies every round through its own update path (Table I); search quality is measured at rounds $0, 4, \ldots, 20$ (SIFT) and $0, 5, 10$ (DEEP).

Parameters. Each method is swept over its search knob until recall exceeds 0.999 or a cap is reached: ACORN `efSearch` 16..2048, Filtered-DiskANN L 16..1024, IVF n_{probe} $8..\min(2048, n_{\text{list}}/2)$ with $n_{\text{list}} = 4\sqrt{n}$, post-filter $k' \in \{100, 400, 1600, 6400\}$ (streams: $\{400, 1600, 6400\}$), RIG ef 16..512; the same ranges apply to the streams. These sweeps, like the dispatcher thresholds of Section IV-C, are evaluated on the reported queries, as is customary in ANN benchmarks; reported operating points are therefore tuned to the query set. Construction parameters follow the original papers' defaults: ACORN M=32, γ=12, M_β=64; Filtered-DiskANN R=64, L=100, α=1.2; HNSW M=32, ef_c=200.

Metrics. Every query asks for k=10 neighbors. Recall@10 of a query is the fraction of its exact filtered top-10 that is returned (over $\min(10, |A_p|)$ when fewer points are admissible), averaged over 500 queries; *maximum recall* is the highest recall in a method's parameter sweep, and QPS at a recall target is interpolated log-linearly on the method's recall–QPS Pareto frontier, except for the dispatcher of Section IV-C, whose QPS is that of the best measured configuration (a concrete threshold and parameter) that reaches the target. QPS is the number of queries divided by the time spent inside the search calls of a single thread.

Measurement protocol. Searches are single-threaded and issue one query per call, with these exceptions: on label workloads ACORN is called once for all 500 queries and RIG once per label (both timed in C++), and post-filtering, the bitmap scan, and the per-label scan are called once per label (the per-label scan is also reported per query); one call per label lets BLAS share work across queries. Some method-specific costs fall on the boundary of the timer: post-filtering's selection of admissible candidates (56 μs per query at k'=6400) and Filtered-DiskANN's check of the second tag of AND queries (73 μs at L=1024) are outside it, as is the Python call into RIG's C++ kernel (354 μs, which favors RIG); ACORN's one-query-per-call mode on YFCC allocates and clears an n-entry visited table inside it (0.40 ms per call, which disfavors ACORN on YFCC). The fixed cost of a Python binding call, timed inside the measurement for the per-query methods, is 12 μs for Faiss, 21 μs for `diskannpy`, and 8 μs for hnswlib, a median 2% of the query time over all measured operating points of these methods and at most 4% at recall ≥ 0.95. The predicate is handed to every method in its native precomputed form (label, bitmap, or admissible list), so predicate evaluation is not timed for any method; on YFCC it is not negligible—intersecting a query's tag lists takes 1.4 ms on average (p99 20.2 ms), and materializing the 10^7-entry mask and packing it into a bitmap another 1.2 and 0.6 ms—the intersection is needed by every method that receives an admissible list or bitmap (including the per-query scan), the mask and bitmap only by the bitmap-filtered methods. Static builds use 32 threads; in the streams, initial builds, update maintenance, and rebuilds use 16 threads, uniformly.

Each configuration is warmed once and timed three times; the median 95% confidence interval is $\pm$0.8% of the mean (90th percentile $\pm$3.8%). The machine, a two-socket Xeon Gold 6258R (2×28 cores, 38.5 MB LLC per socket, 502 GB RAM; Ubuntu 22.04, kernel 6.8, GCC 11.4), was shared with unrelated workloads, so within-job intervals do not capture the dominant source of variance. All jobs ran pinned to one socket (memory allocated there by first touch) and strictly one at a time, and a fixed single-thread calibration kernel ran before and after every job; a job started only when the kernel reached 380 queries per second (its quiet-machine value is 500–600; 350 for the re-measurements of a later, slower session) on two consecutive checks, and was re-run if it ended below that. Every configuration was measured in two passes and we report the faster one; the second pass and every re-run ran under this

gate, whereas part of the first pass predates it (and some of its jobs spanned both sockets), which the best-of-two selection mitigates but does not remove; across the 1073 configurations measured in both passes the median ratio between passes is 1.03 (90th percentile 1.10), and recall is identical across passes for the deterministic methods (the scans, IVF, RIG), differs by at most 0.003 for post-filtering, and differs by a median of 0.003 (90th percentile 0.016) for ACORN-γ, 0.001 (0.006) for Filtered-DiskANN, and 0.007 (0.042) for ACORN-1, whose multi-threaded builds are not deterministic; every reported point keeps the recall and QPS of the same run. Software: Faiss 1.15.1 (AVX2/AVX-512 dispatch), `diskannpy` 0.7.0, hnswlib 0.8.0, and ACORN at commit `c259f11` built with AVX2 and the patch of Section VII.

IV. Static Filtered Search

A. The exact scan is the baseline to beat (*Q1*)

An exact scan over the admissible points costs time proportional to $s \cdot n$ and nothing else; every index must beat it to be worth its build time, memory, and maintenance. To separate the admissible-set size from the collection size, Figure 1 holds $s \cdot n$ fixed at 10^4 and 10^5 and grows n from 10^6 to 10^7 (DEEP subsets, β=0.9), plotting each index's QPS relative to the per-label scan at recall 0.95 and 0.99.

At 10^4 admissible points the scan wins at every n, and by more as n grows. It answers 4,286–2,951 queries per second exactly; the best index, IVF, reaches 0.5× the scan's throughput at recall 0.95 for n=10^6 and only 0.1× for n=10^7, and every graph method fails to reach 0.95 at $n \geq 3.16 \times 10^6$. Curator, which indexes each label's points in its own subtree, is the exception to the decline: relative to the scan it stays at 0.4× for n=10^6 and 0.4× for n=10^7, and at 10^5 admissible points at 1.3× and 1.0×—the most robust index to collection size, though still slower than the scan below 10^5. ACORN's design calls for a larger expansion factor at low selectivity ($\gamma \geq 1/s$) [2]; with γ=100 on SIFT1M at s=0.01, β=0.9, its maximum recall rises only from 0.946 to 0.956, while its build time grows from 181 s to 2,144 s.

At 10^5 admissible points the winner depends on n. At n=10^6 (s=0.1) IVF and Filtered-DiskANN are 3.6× and 3.6× faster than the scan at recall 0.95 (1.4× and 1.3× at 0.99); at n=3.16×10^6 only IVF still wins, by 2.2× at 0.95, and it loses at 0.99; at n=10^7 (s=0.01) IVF merely ties the scan at 0.95 (1.0×) and is 0.4× as fast at 0.99. The scan's cost depends only on how many points it touches. A plausible explanation for the indexes, which we did not instrument, is that their cost also grows with the size of the space they navigate: at lower selectivity a search passes through more inadmissible territory before it collects k admissible neighbors. Curator's nearly constant ratio fits this explanation, since its per-label subtree covers only the label's points. Note that the design varies n and s together, so the two effects are not separated, and that with β=0 IVF still beats the scan at n=10^7 (2.1× at recall 0.95). A crossover stated as a selectivity, or even as an admissible-set size, therefore does not transfer across collection sizes.

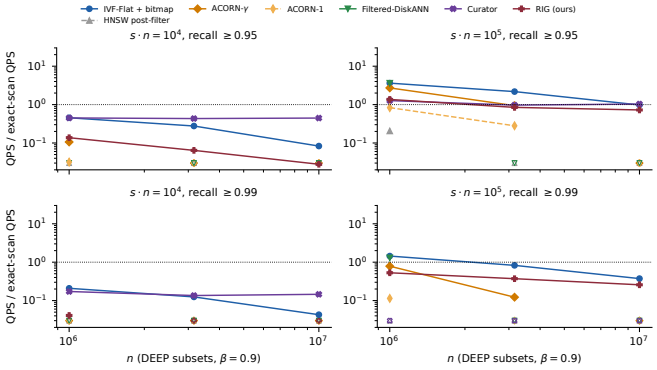


Fig. 1. QPS relative to the exact per-label scan as n grows at a *fixed* admissible-set size $s \cdot n$ (columns: 10^4, 10^5; rows: recall $\geq$ 0.95, $\geq$ 0.99; DEEP subsets, β=0.9). Above the dotted line the index beats the scan; hollow markers on the bottom edge never reach the recall target.

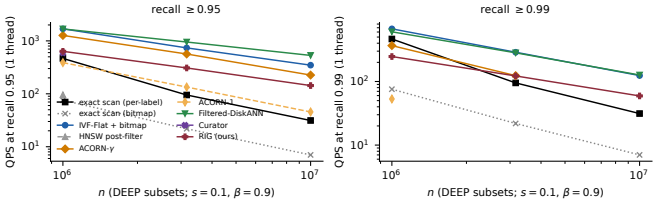


Fig. 2. QPS at recall 0.95 (left) and 0.99 (right) as n grows at fixed selectivity s=0.1 and correlation β=0.9 (DEEP subsets).

At 10^6 admissible points indexes win. Figure 2 shows the regime at fixed s=0.1, where the admissible set grows with n. On DEEP-10M at s=0.1 the scan falls to 31 QPS; at β=0 post-filtering and IVF are 16.2× and 8.1× faster at recall 0.99, and at β=0.9 Filtered-DiskANN and IVF are 4.0× and 4.0× faster. Curator, designed for selective filters, does not reach 0.95 here even at its largest search effort (maximum recall 0.895), consistent with its authors' recommendation to pair it with a graph index for common labels [26].

When the predicate cannot be pre-partitioned, the exact scan must visit all n rows and test the bitmap; this *bitmap scan* answers 7.0 QPS on DEEP-10M versus 31 for the per-label partition, so arbitrary predicates move the crossover toward smaller admissible sets.

B. Correlation reorders the methods (*Q2*)

Figure 3 varies the label–geometry correlation β on SIFT1M at s=0.1. Post-filtering is among the fastest methods when labels are independent of geometry—854 QPS at 0.99 for β=0 on SIFT1M, within a few percent of IVF (859), and the fastest of all on DEEP-10M (506)—and the worst when they are not: its maximum recall falls to 0.903 at β=0.9 and 0.897 at β=1, because a query's unfiltered neighborhood then contains few admissible points. Filtered-DiskANN moves the other way—from 290 to 913 QPS at 0.99—because its per-label medoids and label-preserving edges sit in the region where a correlated label lives. ACORN and IVF lose throughput as correlation grows (ACORN 721→243, IVF 859→489): for

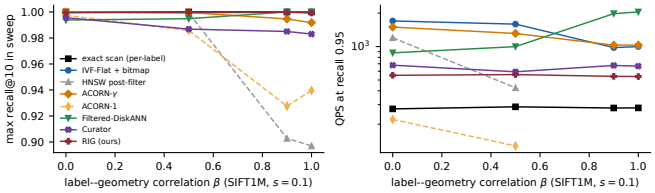


Fig. 3. Effect of label–geometry correlation on SIFT1M (s=0.1): maximum recall reached in each method's sweep (left) and QPS at recall 0.95 (right).

IVF the admissible points of a query's label live in lists far from the query, so more lists must be probed. Curator is insensitive to correlation (maximum recall 0.996 to 0.983, 680 to 667 QPS at 0.95), as a per-label index should be. On both SIFT1M and DEEP-10M, a benchmark that fixes β=0—uniformly random attributes, the common choice [8]—would thus favor post-filtering and penalize the label-specialized graph.

C. Real predicates need a dispatcher (*Q3*)

YFCC-10M's queries span four orders of magnitude of selectivity, and no index reaches recall 0.99 on them by itself. Table II breaks recall down by admissible-set size. The graph methods fail on small admissible sets: ACORN's recall is 0.027 below 10^3 admissible points and 0.190 between 10^3 and 10^4, and reaches 0.995 only above 10^5; RIG, whose entry points are a 1% sample, finds no admissible entry for the most selective queries (0.460); Filtered-DiskANN, which can follow only one tag of an AND and whose rare-tag subgraphs are poorly connected (Section VII), stays below 0.632 except on the most common tags. IVF degrades least (0.956 below 10^3). Meanwhile the per-query scan answers the smallest bucket at 5,608 QPS.

A common remedy is a selectivity-based dispatcher; ACORN's authors propose pre-filtering whenever a query's estimated selectivity is below $1/\gamma$ [2]. We evaluate the oracle version: queries whose admissible set is smaller than a threshold T are answered by the exact scan, the rest by the index, and T and the index's parameter are chosen jointly to maximize QPS at a recall target (Table II, right). With an oracle dispatcher every method reaches 0.99; IVF is fastest (311 QPS at T=32,768), ahead of ACORN (231), post-filtering (200), and RIG (141). For IVF, ACORN-γ, post-filtering, and RIG the optimal thresholds lie between 32,768 and 65,536 admissible points, the same order as the crossover of Section IV-A at n=10^7; for the weaker ACORN-1 and Filtered-DiskANN they are higher (262,144 and 524,288). To check that a tuned dispatcher transfers, we drew 500 further public YFCC queries disjoint from the first set (their exact ground truth also matches the official one) and applied to them, unchanged, the threshold and parameter chosen on the original queries. At the 0.99 target their recall is 0.984–0.994 across methods (IVF 0.994, ACORN 0.984), so the oracle's choices generalize up to a small recall loss, and the methods keep the oracle's order: IVF answers 228 QPS, ACORN 176, post-filtering 144, and RIG 113, with p99 latencies of 14.2 ms for IVF and 30.4 ms for ACORN. (The held-out queries were measured in a later session on a slower machine state, so their throughputs are comparable with each other but not with the oracle column.) Curator has the opposite profile to the graph indexes: it is exact on selective queries (recall 1.000 below 10^3 admissible points and 1.000 between 10^3 and 10^4) and weaker on common ones (0.965 between 10^5 and 10^6). Dispatching between it and the scan therefore helps little (38 QPS at 0.99), but dispatching between it and a graph or IVF—its authors' recommended use [26]—is the fastest configuration we found: Curator below T=65,536 admissible points and IVF above reach 0.99 at 368 QPS, and Curator with ACORN at 346, ahead of the best scan-based dispatcher (IVF, 311); on the disjoint held-out queries the two pairings keep their lead (312 and 315 QPS at recall 0.989 and 0.985, versus 228 for scan plus IVF). An index's YFCC result without a dispatcher is therefore dominated by its failures on selective queries, which a dispatcher routes to the scan.

TABLE II

YFCC-10M (REAL TAGS). LEFT: RECALL BY ADMISSIBLE-SET SIZE $|A|$ AT EACH METHOD'S HIGHEST-RECALL SETTING (QUERIES PER BUCKET IN PARENTHESES). RIGHT: QPS AT RECALL ≥ 0.99 WITH A DISPATCHER THAT SCANS QUERIES WITH $|A| < T$—ORACLE (THRESHOLD AND PARAMETER CHOSEN ON THESE QUERIES) AND HELD-OUT (THE SAME CHOICE APPLIED TO 500 DISJOINT QUERIES, MEASURED IN A LATER SESSION; THEIR RECALL IN PARENTHESES)—AND THE HELD-OUT P99 LATENCY IN MS.

	recall by $\lvert A\rvert$					dispatched, recall ≥ 0.99		
	$<10^3$ (123)	10^3–10^4 (112)	10^4–10^5 (113)	10^5–10^6 (116)	$\geq 10^6$ (36)	oracle QPS	held-out queries QPS (rec.)	p99 ms
IVF	0.956	0.998	0.999	1.000	1.000	311	228 (0.994)	14.2
ACORN-γ	0.027	0.190	0.777	0.995	1.000	231	176 (0.984)	30.4
ACORN-1	0.015	0.189	0.748	0.980	0.989	82	62 (0.989)	121.1
post-filter	0.435	0.721	0.932	0.997	0.997	200	144 (0.993)	30.4
F-DiskANN	0.093	0.179	0.122	0.632	0.975	37	33 (0.989)	261.3
Curator	1.000	1.000	0.998	0.965	0.953	38	34 (0.988)	261.3
RIG	0.460	0.895	0.991	0.991	0.950	141	113 (0.992)	43.3
scan (per query)	1.000	1.000	1.000	1.000	1.000	11	—	—

D. Build cost and memory

The methods differ by up to 41× in build time and 10× in memory (Table III). IVF builds DEEP-10M in 60 s at the size of the data; ACORN needs 2,109 s and Filtered-DiskANN 1,331 s on the same 32 threads, and Filtered-DiskANN's build is per label schema, so a new attribute means a new index. RIG's streaming layout duplicates codes across pools and is the largest index (39.6 GB on DEEP-10M). ACORN-1, the variant without neighbor expansion, builds 22–23× faster than ACORN-γ (94 s on DEEP-10M) but trades recall for it: its maximum recall at s=0.1, β=0.9 is 0.928 on SIFT1M and 0.967 on DEEP-10M, versus 0.995 and 0.985 for ACORN-γ.

E. Multi-threaded throughput

All throughputs so far are single-threaded, which isolates per-query cost but not how a method uses a socket. Table IV repeats three workloads with 28 search threads on the 28 physical cores of one socket. Each method runs in its native batched mode—one call per label for the scans, IVF, and RIG,

TABLE III
BUILD TIME (S, 32 THREADS) AND INDEX SIZE (GB) AT $n = 10^7$.

	DEEP-10M build	DEEP-10M GB	YFCC-10M build	YFCC-10M GB
exact scan	—	3.9	—	7.7
IVF-Flat	60	3.9	80	7.8
HNSW (post-filter)	457	6.6	1,013	10.4
ACORN-γ	2,109	8.9	3,273	12.8
ACORN-1	94	7.2	148	11.0
Filtered-DiskANN*	1,331	10.3	1,844	19.5
Curator§	229	4.2	392	18.7
RIG	181	39.6	333	73.1

*Per label schema. §Size measured as resident-memory growth; per label schema; on YFCC only the queried tags are indexed (int16 label ids).

TABLE IV
QPS AT RECALL 0.95 WITH 1 AND 28 SEARCH THREADS, MEASURED IN THE SAME SESSION (SCALING IN PARENTHESES; N/R: NOT REACHED). *THREAD POOL OF PER-QUERY CALLS; THE BINDING HAS NO BATCHED FILTERED SEARCH.

	DEEP-10M, β=0		DEEP-10M, β=0.9		SIFT1M, β=0.9	
	1 thr.	28 thr.	1 thr.	28 thr.	1 thr.	28 thr.
exact scan (per-label)	29	289 (9.8×)	29	294 (10.1×)	286	1,533 (5.4×)
IVF-Flat	664	9,083 (13.7×)	338	4,117 (12.2×)	1,042	11,399 (10.9×)
post-filter	884	14,760 (16.7×)	n/r	n/r (—)	n/r	n/r (—)
ACORN-γ	532	9,245 (17.4×)	218	3,518 (16.1×)	911	15,489 (17.0×)
Filtered-DiskANN*	295	178 (0.6×)	522	330 (0.6×)	1,908	1,077 (0.6×)
RIG	161	1,157 (7.2×)	148	1,011 (6.8×)	517	3,058 (5.9×)

one call for all queries for ACORN and hnswlib—and the scans, IVF, and RIG also run query-parallel, with 28 threads issuing single-query, single-threaded calls; we report the better mode (batched won wherever both ran; query-parallel Faiss calls at 10^7 points abort inside the OpenBLAS bundled with `faiss-cpu`), the better of two passes, and scaling against single-thread runs from the same session, because the machine's single-thread speed differed between sessions. ACORN parallelizes over queries in C++ and scales 16.1–17.4×, post-filtering 16.7×, IVF 10.9–13.7×, the batched scan 5.4–10.1×, and RIG 5.9–7.2×. The ranking changes with the workload: with β=0 on DEEP-10M post-filtering is the fastest method at 28 threads (14,760 QPS at recall 0.95), with ACORN and IVF tied behind it (9,245 and 9,083); with β=0.9 IVF stays ahead of ACORN (4,117 versus 3,518); and on SIFT1M ACORN overtakes IVF (15,489 versus 11,399). The Python binding of Filtered-DiskANN exposes no batched filtered search, and a thread pool of per-query calls is *slower* than one thread, so its single-threaded throughput is also its best. Which index is fastest therefore depends on whether queries are served one at a time on one core or in batches across all cores; the update-stream comparisons that follow are single-threaded.

V. FILTERED SEARCH UNDER UPDATES (**Q4**)

Figure 4 follows each method through the three streams, and Table V summarizes the last round. We group the methods by the update paths of Table I.

Exact deletion (scan, IVF, RIG). The per-label scan and IVF delete by removing entries, so nothing accumulates. IVF holds a maximum recall of 1.000 on SIFT-500K after 200% turnover and 1.000 on DEEP-5M after 100%, at nearly unchanged throughput (2,444→2,321 and 557→546 QPS at recall 0.95), for a total maintenance of 1.2 s and 12.4 s—the cheapest of any index (Curator needs 5–9× as much, and the graph indexes two to three orders of magnitude more). RIG also deletes exactly and keeps its maximum recall at or above 0.998 in every round of every stream, but its compaction costs 38–39× IVF's maintenance and its throughput at recall 0.95 is about a third of IVF's.

Curator deletes by revoking a point's labels, which removes it from the per-label subtrees although its vector stays in the shared tree; its quality also stays flat (maximum recall 0.987 on SIFT-500K and 0.990 under correlated deletions, 0.916 on DEEP-5M, where it never reaches 0.95), at 5–9× IVF's maintenance.

Deletion through the predicate (ACORN, post-filtering). ACORN has no deletion routine; deleted points stay in the graph and are excluded by the query bitmap. Its graph therefore grows (3.0× resident nodes on SIFT-500K) and every deletion lowers the density of live admissible points that in-filter navigation must traverse. Under uniform deletions its maximum recall drifts from 1.000 to 0.993 and its throughput at recall 0.95 from 1,855 to 656; on DEEP-5M it reaches 0.99 at round 0 but not at the later measured rounds (5 and 10), and its throughput at 0.95 falls from 427 to 197, while its incremental insertions cost 119× IVF's maintenance. *Correlated* deletions are far worse: when the stream deletes the geometric region of the highest labels (SIFT-500K-adv), ACORN's maximum recall collapses from 1.000 to 0.970 after four rounds and 0.575 after twenty. A control separates the update path from the data: rebuilt from scratch on exactly the live points, labels, and queries of round 20, ACORN reaches a maximum recall of 0.995 (1,300 QPS at recall 0.95) on the correlated-deletion stream and 1.000 on the uniform one, against 0.575 and 0.993 for the incrementally maintained graph. The collapse and the drift are therefore caused by the deleted nodes that ACORN's update path keeps in the graph, not by the changed label distribution; plausibly, the dead nodes form inadmissible regions that in-filter navigation cannot cross. (The fresh builds ran in a later session on a slower machine state, so their throughput is, if anything, understated.) Post-filtering with slot reuse keeps its recall under uniform churn but is slow at high recall (273 QPS at 0.95 on SIFT-500K) and capped by the correlated predicate on DEEP-5M (maximum recall 0.913); under correlated deletions it loses ground (0.990→0.964).

Rebuild (Filtered-DiskANN). Freshly rebuilt, Filtered-DiskANN is the fastest method at recall 0.95 in the last round of all three streams (2,490 QPS on SIFT-500K, 828 on DEEP-5M, 3,771 on SIFT-500K-adv; at 0.99, 948 versus IVF's 1,188 on SIFT-500K), but each rebuild takes 52 s and 820 s on 16 threads, so staying fresh every round costs 1,043 s and 8,197 s—861× and 659× IVF's maintenance. Between rebuilds, inserted points are invisible to it.

Scan throughput. The scan stays exact under every stream, but its throughput tracks the live admissible-set sizes: under

TABLE V
LAST ROUND OF EACH STREAM: MAXIMUM RECALL / QPS AT RECALL 0.95 (1 THREAD; N/R: NOT REACHED) / TOTAL MAINTENANCE (S, 16 THREADS; FILTERED-DISKANN: ONE REBUILD PER ROUND).

	SIFT-500K, round 20			DEEP-5M, round 10			SIFT-500K-adv, round 20		
	rec.	QPS	maint.	rec.	QPS	maint.	rec.	QPS	maint.
exact scan	1.000	859	2.7	1.000	62	16.7	1.000	550	2.5
IVF	1.000	2,321	1.2	1.000	546	12.4	0.999	1,866	1.2
post-filter	0.991	273	246.0	0.913	n/r	1,565.4	0.964	109	208.7
ACORN-γ	0.993	656	276.1	0.982	197	1,481.1	0.575	n/r	275.1
F-DiskANN	0.999	2,490	1,043	0.996	828	8,197	1.000	3,771	1,084
Curator	0.987	885	9.7	0.916	n/r	57.5	0.990	1,498	11.1
RIG	1.000	724	46.1	0.999	202	486.5	0.999	578	46.1

correlated deletions, labels whose region is emptied shrink and others grow, and the scan's QPS falls from 864 to 550.

VI. TOTAL COST (**Q5**)

A deployment pays for maintenance and for queries. Per round of U updates and Q filtered queries, method X costs $M_X + Q/\mathrm{QPS}_X$, where M_X is its maintenance time and QPS_X its single-thread throughput at the required recall. On all three streams IVF is cheaper than ACORN, post-filtering, Curator, and RIG on *both* terms at recall 0.95, so with single-threaded queries those methods are dominated for every ratio Q/U. With all cores serving queries the per-query term can favor ACORN (Section IV-E), but its maintenance term stays 119–228$\times$ IVF's, so it pays off only for query-dominated streams—and its recall still drifts. Only a freshly rebuilt Filtered-DiskANN is faster per query; it saves 0.62 ms per query on DEEP-5M and 0.03 ms on SIFT-500K, and recovers the cost of one rebuild per round only above 1,311,450 queries per round on DEEP-5M (with 998,651 updates per round), i.e. when there are more than 1.3 queries per update counting wall time, or 21 counting the rebuild's 16 threads as CPU time (SIFT-500K: 17.8 and 285). These break-even points are unstable, because they divide a large maintenance difference by a small per-query difference: computed at each measured round they span 1.3–2.2 queries per update on DEEP-5M and 17.8–38.4 on SIFT-500K, where IVF is faster per query in 2 of 6 rounds and Filtered-DiskANN never pays off; at recall 0.99 IVF is faster per query in every round of SIFT-500K. Correlated deletions shift the balance toward Filtered-DiskANN, because IVF's throughput falls under these deletions (2,514→1,866 QPS at 0.95) while a fresh rebuild does not degrade: on SIFT-500K-adv the per-round break-even spans 1.8–207.9 queries per update at 0.95 (the largest value at round 0, before the deletions take effect, where the two differ by under 1%) and 0.9–2.1 at 0.99 (IVF is faster per query in 1 of 6 rounds), so in the late rounds a rebuild per round pays off with fewer queries than updates. Rebuilding less often lowers the maintenance term but leaves new points invisible to filtered queries in the meantime; our cost model does not price this staleness.

Memory follows the update path. IVF and the scan hold the live data plus identifiers (1.96 GB on DEEP-5M); Filtered-DiskANN and HNSW add a graph (5.13 and 3.63 GB); ACORN's graph grows with every insertion because deleted nodes are never removed (4.47→8.93 GB); RIG's streaming layout is the largest (19.83 GB).

VII. LESSONS FOR EVALUATING FILTERED SEARCH (**Q6**)

Running nine methods from six code bases on a shared machine surfaced failure modes of the *evaluation* as much as of the methods; we report them because each one distorted reported numbers before we caught it, and contention reversed a conclusion.

Machine contention. Our calibration kernel varied between 6 and 633 queries per second over the 14 days of our measurements as unrelated workloads came and went, a $100\times$ range, while the variation between repetitions inside a job was a few percent. Repetitions and confidence intervals therefore do not protect a comparison whose arms ran at different times: in one early pass, the scan and IVF arms of seven SIFT workloads ran while the calibration kernel stood at a quarter of its quiet-machine value; re-run on a quiet machine, the same configurations were up to $2.9\times$ faster (IVF at recall 0.99 on SIFT1M with s=0.01, β=0.9: from 242 to 696 QPS; with s=0.1: from 204 to 482), enough to reverse which method wins at $s \cdot n = 10^5$. We now pin every job to one socket, run jobs strictly sequentially, gate each job on the calibration kernel (a job starts only when the kernel reaches 380 queries per second, about two thirds of its quiet-machine value, and is re-run if it ends below that; 350 for the re-measurements of the last session, whose quiet-machine value was lower), and take the better of two passes.

Defects in baseline implementations. ACORN's incremental `add` crashed our first stream run at the first insertion round: when an upper-level neighbor list is full, the original code writes the merged candidate set back without truncating it, one entry past the node's range, which corrupts the adjacency array. We truncate to capacity, as upstream Faiss HNSW does (a three-line patch in our artifacts), and use the patched library for every ACORN measurement, static and dynamic; since the overflow also occurs during bulk construction whenever an upper-level list is full, unpatched static builds are affected as well. ACORN's documented build compiles *scalar* distance kernels: its CMake script declares the optimization level with `option()`, which turns the intended string into a boolean, so the vectorized Faiss kernels are never selected. Rebuilt with `-DFAISS_OPT_LEVEL=avx2`, ACORN built $1.5\times$ faster and searched 1.1–$1.4\times$ faster on SIFT1M at identical recall; all ACORN numbers in this paper use the vectorized build. `diskannpy` pads results with *uninitialized* identifiers when a filtered search finds fewer than k matches; counting them, or even filtering them by identifier range, inflates or deflates recall depending on memory contents, so every returned identifier must be validated against the predicate.

A documented Filtered-DiskANN limitation. Filtered-DiskANN with one label per point reaches recall ≈ 0.99 at s=0.1 but stays below 0.2 at s=0.01 on both SIFT1M and DEEP-10M, and larger search lists barely help (with β=0 recall is identical from L=16 to L=1024; with β=0.9 it does

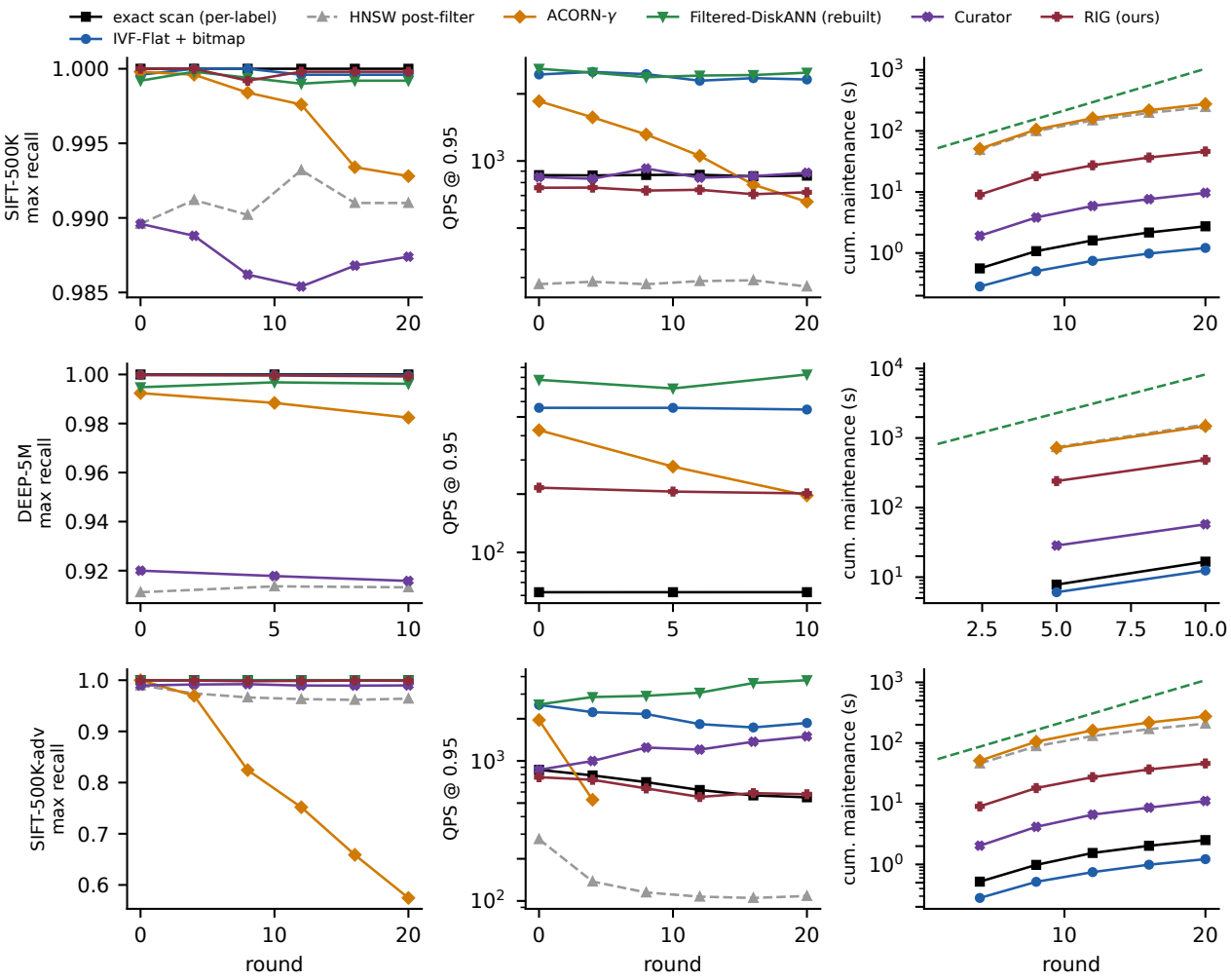


Fig. 4. Filtered search under updates (s=0.1, β=0.9). Rows: SIFT-500K (20 rounds of 10% uniform deletions and 5×10^4 insertions), DEEP-5M (10 rounds, 100% turnover), SIFT-500K-adv (deletions concentrated on the label direction). Columns: maximum recall in each method's sweep, QPS at recall 0.95 (missing where 0.95 is not reached), and cumulative maintenance on 16 threads (for Filtered-DiskANN, the cost of one rebuild per round).

not exceed 0.168 on SIFT1M and 0.174 on DEEP-10M): the search exhausts the part of the label's subgraph reachable from its medoid. We reproduced this on synthetic data with uniform labels and found the same behavior reported for the original C++ tools (recall plateauing near 0.5 at 5% specificity, independent of L [38]) and in recent benchmarks [26], [9]. We report it as a property of the public implementation rather than of the algorithm.

Duplicate vectors. Our first SIFT stream re-inserted SIFT1M vectors under fresh identifiers to reach 1.5×10^6 rows; once a re-inserted copy and its original were both live, a method returning the copy was charged a miss. The streams reported here draw 1.5×10^6 distinct SIFT descriptors from BIGANN instead.

Generated numbers. Every number in this paper that is derived from the final benchmark results is a macro generated from the analysis output, so that re-running an experiment cannot leave a stale value in the text; the few numbers quoted from superseded runs (the contended pass, the ACORN build comparison) are stated as such.

Simple baselines. Each simple component changed a conclusion here: the exact scan is the fastest method at $s \cdot n \leq 10^4$ (Section IV-A); IVF with a bitmap is competitive on the static workloads and the cheapest index to maintain on every stream; and on YFCC no index reaches recall 0.99 without a dispatcher. We therefore recommend that filtered-search evaluations report all three.

VIII. Limitations and Threats to Validity

Measurement. All results come from one shared machine. The gating protocol of Section III removes runs taken under gross contention, but part of the first pass predates the gate, and differences below the pass-to-pass spread (90th percentile 1.10×) should not be over-read; this includes the closest 28-thread comparisons (Table IV), such as ACORN versus IVF on DEEP-10M with β=0. The machine's single-thread speed also

varied between sessions (the calibration kernel's quiet value fell by about a third in the last session, plausibly through reduced turbo headroom); every ratio we report is therefore formed within one session, and throughputs from different sessions are compared only by their ranking. Each workload has 500 queries; the standard error of mean recall, computed from the per-query recalls, has a median of 0.0017 and a maximum of 0.009 for configurations at recall ≥ 0.95, and reaches 0.022 for the low-recall YFCC configurations.

Tuning on the reported queries. Parameter sweeps and the dispatcher's threshold are evaluated on the queries whose results we report, as is customary for ANN benchmarks; applied unchanged to 500 disjoint YFCC queries, the dispatcher's choices reach a recall of 0.984–0.994 at the 0.99 target. Reported operating points are therefore optimistic for every method alike.

Workloads. The synthetic labels are quantiles of one random projection, one label per point, with correlation controlled by a single parameter; real multi-tag predicates come from one dataset (YFCC-10M); all queries ask for k=10. Insertions in our streams follow the distribution of the base data, so IVF's centroids, fixed after the initial training, were never stressed by distribution drift; streams with drifting insertions could reverse the update-side conclusions for IVF. Deletions are either uniform or concentrated along one direction.

What is not timed. Predicate evaluation is excluded for every method. On YFCC it costs 3.2 ms per query on average (intersecting tag lists, materializing the mask, and packing the bitmap)—about as much as the dispatched IVF search itself (99%)—and the bitmap-filtered methods (IVF, ACORN, post-filtering, RIG) pay all of it, while the per-query scan and Curator's AND queries pay the intersection and Filtered-DiskANN, which filters on one tag, pays the check of the second (outside the timer, Section III). Charging the full cost to every query of the oracle-dispatched configurations at recall 0.99 would lower their throughput to 50% (IVF) to 69% (RIG) of the reported values; queries routed to the scan pay only the intersection, so this bounds the effect.

Implementations and coverage. We evaluate public implementations through their native or Python interfaces. Filtered-DiskANN is evaluated through `diskannpy`, which exposes neither a dynamic filtered index nor parallel filtered search, and its low-specificity failure is a property of that implementation; the C++ library may offer more. ACORN is evaluated with our overflow patch and an AVX2 build, with γ=12 except for one γ=100 test. We do not evaluate range-filter structures, label-centric indexes without an update path (UNG, NHQ, CAPS, SIEVE), the dynamic filtered index EMA, or vector database systems; Curator is evaluated with its authors' YFCC parameters and the update path its released code provides; disk-resident indexes and concurrent query–update execution are out of scope.

Explanations and scope of claims. One explanation in this paper is a hypothesis we did not instrument: why the indexes' advantage shrinks with n at fixed $s \cdot n$ (Curator's behavior is consistent with it, Section IV-A); ACORN's collapse under correlated deletions, by contrast, is attributed to its update path by a fresh-build control (Section V). The study also began as the evaluation of a new index; its questions were formulated after that evaluation showed simple baselines to be competitive, and its findings should be read as an exploratory characterization rather than as confirmatory tests.

Conflict of interest. RIG is the author's own, unpublished design. It was developed on SIFT1M and DEEP with the same query sets used here, which may favor it slightly; we include it as a design point, report its losses throughout, and draw no conclusion that depends on it.

IX. Related Work

Experimental studies of filtered search. The closest studies evaluate filtered search on *static* data. Zhu et al. [7] evaluate fifteen hybrid-query algorithms for attribute and range filtering, with standardized attribute and range sets, varying data distributions and platforms, and a 10^8-point scalability test; dynamic insertion appears in their taxonomy as a capability, but their experiments, like the others below, use static data and do not control the correlation between attributes and vectors. Li et al. [8] compare twelve attribute-filtering methods, including SeRF, iRangeGraph, Filtered-DiskANN, NHQ, ACORN, and Faiss/Milvus HNSW and IVF-PQ, on up to 10^7 vectors with uniform synthetic attributes, and list dynamic updates and vector–attribute correlation as open problems. Shi et al. [9] benchmark ten label-filter methods on six real datasets with unified tuning and include pre-filter brute force and post-filtered IVF-PQ. Iff et al. [10] contribute a dataset of transformer embeddings with eleven attributes. Lu et al. [11] study filter-agnostic inline methods inside a PostgreSQL-compatible engine with a workload generator that controls correlation, and show that crossover points move once methods run inside a DBMS; Amanbayev et al. [12] compare Faiss, Milvus, and pgvector and find IVF-Flat ahead of HNSW at low selectivity; Lim et al. [13] argue that selectivity and correlation are unstable proxies for hardness. Sanca and Ailamaki [14] characterize scan-versus-index access paths for mixed queries. The filtered track of Big-ANN'23 [5], whose YFCC data we reuse, already switched between brute force over the matching subset and an index by tag cardinality. Chronis et al. [15] survey the area in a tutorial. Relative to these, we do not claim the first scan or IVF baseline, nor the first controlled correlation; our contributions are the joint treatment of filters *and* updates, and stating the crossover in absolute admissible-set size across n and under churn.

Streaming ANN. The Big-ANN'23 streaming track evaluates insert/search/delete runbooks without filters, and CANDY [16] benchmarks continuous ingestion. Index papers evaluate their own update paths: FreshDiskANN [6], SPFresh [17], in-place DiskANN updates [18], and incremental IVF maintenance [19]; Xiao et al. [20] document unreachable points in HNSW under replaced updates, and Yamashita et al. [21] formalize how deletion in graph indexes should be evaluated. Dynamic *filtered* indexes exist—DIGRA [22], the dynamic range-filter index of Zhang et al. [23], DSG [24],

WoW [25], Curator [26] (which we evaluate), and EMA [27]—but each is evaluated on its own update path, against static baselines or on update latency; we know of no study that compares several filtered methods, together with exact-scan and IVF baselines, while the data churns, or that varies how deletions correlate with the attribute.

Filtered-search methods not evaluated here. Range-filter structures (SeRF [28], iRangeGraph [29], UNIFY [30], window filters [31]) target range predicates; label-centric methods include NHQ [32], UNG [33], CAPS [34], and SIEVE [35]; NaviX [36] and Compass [37] target general predicates inside database systems. We restrict the study to methods with a native or documented update path and a public implementation that accepts arbitrary or label predicates, plus the scan and IVF baselines.

X. Conclusion

Our measurements suggest four guidelines. *(i)* Keep an exact pre-filter scan—over label-partitioned data where the predicate allows it, since a bitmap scan of all rows is several times slower—and dispatch to it below roughly 10^4–10^5 admissible points (our range for recall ≥ 0.95), setting the threshold for the collection size at hand rather than for a selectivity; where a per-label index such as Curator is available, it is a better choice than the scan for the selective side of the dispatcher, and IVF or a graph for the common side. *(ii)* Measure the correlation between attributes and embeddings before choosing between post-filtering and a filter-aware index; on our data, attributes independent of the embedding favored post-filtering. *(iii)* Under churn, prefer indexes that delete exactly: a bitmap-filtered IVF index was accurate, cheap to maintain, and fast across our streams, whose insertions follow the base distribution and so never stressed its fixed centroids, while deletion through the predicate degraded an in-filter graph sharply when deletions correlated with the attribute. *(iv)* Rebuild a label-specialized graph only if the ratio of queries to updates exceeds the break-even our cost model gives for the stream and recall target. For researchers, the open problem is an in-filter graph with a real deletion path whose quality survives correlated churn, and a dispatcher that adapts its threshold to n and to the update stream.

These conclusions are bounded by the setup and the threats discussed in Section VIII.

Acknowledgment

Use of AI-generated content. In accordance with the IEEE policy on AI-generated content, we disclose that an AI system, Claude Opus 5.5 (Anthropic), used through the Claude Code agent, was used substantially in this work. It implemented the benchmark harness, the workload generators, the drivers and patches for the baselines, and the regenerable pool index RIG (Sections II–III); it ran the experiments and analyzed their results (Sections IV–VI); it generated every figure and table, through scripts that read the raw results; it identified the implementation defects reported in Section VII; it drafted the text of all sections; and AI agents performed internal reviews and an audit of the manuscript, code, and results. The human author defined the research questions, made the design and scope decisions, reviewed every result and revision, and takes full responsibility for the content. No AI system is listed as an author.

Artifacts. The benchmark harness, workload generators, build scripts and patches for the baselines, raw per-query results, and the scripts that produce every number, table, and figure are available at https://github.com/suanlab/filtered-ann-updates-bench; its `REPRODUCE.md` gives the complete pipeline.

References

[1] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 42, no. 4, pp. 824–836, 2020, doi: 10.1109/TPAMI.2018.2889473.

[2] L. Patel, P. Kraft, C. Guestrin, and M. Zaharia, "ACORN: Performant and predicate-agnostic search over vector embeddings and structured data," *Proc. ACM Manag. Data*, vol. 2, no. 3, pp. 1–27, 2024, doi: 10.1145/3654923.

[3] S. Gollapudi et al., "Filtered-DiskANN: Graph algorithms for approximate nearest neighbor search with filters," in *Proc. ACM Web Conf. (WWW)*, 2023, pp. 3406–3416, doi: 10.1145/3543507.3583552.

[4] M. Douze et al., "The Faiss library," *IEEE Trans. Big Data*, vol. 12, no. 2, pp. 346–361, 2026, doi: 10.1109/TBDATA.2025.3618474.

[5] H. V. Simhadri et al., "Results of the Big ANN: NeurIPS'23 competition," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 38, 2025, pp. 76394–76408, doi: 10.52202/085713-2303.

[6] A. Singh, S. J. Subramanya, R. Krishnaswamy, and H. V. Simhadri, "FreshDiskANN: A fast and accurate graph-based ANN index for streaming similarity search," arXiv:2105.09613, 2021.

[7] J. Zhu et al., "An experimental evaluation of hybrid querying on vectors," *Proc. VLDB Endow.*, vol. 19, no. 2, pp. 183–195, 2025, doi: 10.14778/3773749.3773757.

[8] M. Li, X. Yan, B. Lu, Y. Zhang, J. Cheng, and C. Ma, "Attribute filtering in approximate nearest neighbor search: An in-depth experimental study," *Proc. ACM Manag. Data*, vol. 3, no. 6, pp. 1–26, 2025, doi: 10.1145/3769763.

[9] J. Shi, Y. Cai, and W. Zheng, "Filtered approximate nearest neighbor search: A unified benchmark and systematic experimental study," arXiv:2509.07789, 2025.

[10] P. Iff, P. Brügger, M. Chrapek, D. Kochergin, M. Besta, and T. Hoefler, "Benchmarking filtered approximate nearest neighbor search algorithms on transformer-based embedding vectors," in *Proc. 49th Int. ACM SIGIR Conf. Res. Develop. Inf. Retrieval (SIGIR)*, 2026, pp. 610–622, doi: 10.1145/3805712.3809731.

[11] D. Lu et al., "An in-depth study of filter-agnostic vector search on a PostgreSQL database system," *Proc. ACM Manag. Data*, vol. 4, no. 3, pp. 1–26, 2026, doi: 10.1145/3802011.

[12] A. Amanbayev, B. Tsan, T. Dang, and F. Rusu, "Filtered approximate nearest neighbor search in vector databases: System design and performance analysis," arXiv:2602.11443, 2026.

[13] M. Lim, D. Kim, M. Kim, and J. Do, "Revisiting filtered ANN benchmarks: A hardness-controlled benchmark generator for realistic evaluation," *Proc. VLDB Endow.*, vol. 19, no. 10, pp. 2763–2776, 2026, doi: 10.14778/3828612.3828630.

[14] V. Sanca and A. Ailamaki, "Efficient data access paths for mixed vector-relational search," in *Proc. 20th Int. Workshop Data Manage. New Hardware (DaMoN)*, 2024, pp. 1–9, doi: 10.1145/3662010.3663448.

[15] Y. Chronis, H. Caminal, Y. Papakonstantinou, F. Özcan, and A. Ailamaki, "Filtered vector search: State-of-the-art and research opportunities," *Proc. VLDB Endow.*, vol. 18, no. 12, pp. 5488–5492, 2025, doi: 10.14778/3750601.3750700.

[16] X. Zeng, Z. Wu, X. Hu, X. Shi, S. Sun, and S. Zhang, "CANDY: A benchmark for continuous approximate nearest neighbor search with dynamic data ingestion," arXiv:2406.19651, 2024.

[17] Y. Xu et al., "SPFresh: Incremental in-place update for billion-scale vector search," in *Proc. 29th ACM Symp. Operating Syst. Princ. (SOSP)*, 2023, pp. 545–561, doi: 10.1145/3600006.3613166.

[18] H. Xu, M. D. Manohar, P. A. Bernstein, B. Chandramouli, R. Wen, and H. V. Simhadri, "In-place updates of a graph index for streaming approximate nearest neighbor search," arXiv:2502.13826, 2025.

[19] J. Mohoney et al., "Incremental IVF index maintenance for streaming vector search," arXiv:2411.00970, 2024.

[20] W. Xiao, Y. Zhan, R. Xi, M. Hou, and J. Liao, "Enhancing HNSW index for real-time updates: Addressing unreachable points and performance degradation," arXiv:2407.07871, 2024.

[21] T. Yamashita, D. Amagata, and Y. Matsui, "How should we evaluate data deletion in graph-based ANN indexes?" presented at the *NeurIPS 2025 Workshop on Machine Learning for Systems*, 2025, arXiv:2512.06200.

[22] M. Jiang et al., "DIGRA: A dynamic graph indexing for approximate nearest neighbor search with range filter," *Proc. ACM Manag. Data*, vol. 3, no. 3, pp. 1–26, 2025, doi: 10.1145/3725399.

[23] F. Zhang et al., "Efficient dynamic indexing for range filtered approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 3, no. 3, pp. 1–26, 2025, doi: 10.1145/3725401.

[24] Z. Peng, M. Qiao, W. Zhou, F. Li, and D. Deng, "Dynamic range-filtering approximate nearest neighbor search," *Proc. VLDB Endow.*, vol. 18, no. 10, pp. 3256–3268, 2025, doi: 10.14778/3748191.3748193.

[25] Z. Wang, J. Zhang, and W. Hu, "WoW: A window-to-window incremental index for range-filtering approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 3, no. 6, pp. 1–27, 2025, doi: 10.1145/3769843.

[26] Y. Jin et al., "Curator: Efficient vector search with low-selectivity filters," *Proc. ACM Manag. Data*, vol. 4, no. 1, pp. 1–27, 2026, doi: 10.1145/3786635.

[27] M. Li, B. Lu, J. Cheng, and C. Ma, "EMA: Approximate nearest neighbor search with general attribute filtering and dynamic updates," arXiv:2606.00734, 2026.

[28] C. Zuo, M. Qiao, W. Zhou, F. Li, and D. Deng, "SeRF: Segment graph for range-filtering approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 2, no. 1, pp. 1–26, 2024, doi: 10.1145/3639324.

[29] Y. Xu, J. Gao, Y. Gou, C. Long, and C. S. Jensen, "iRangeGraph: Improvising range-dedicated graphs for range-filtering nearest neighbor search," *Proc. ACM Manag. Data*, vol. 2, no. 6, pp. 1–26, 2024, doi: 10.1145/3698814.

[30] A. Liang, P. Zhang, B. Yao, Z. Chen, Y. Song, and G. Cheng, "UNIFY: Unified index for range filtered approximate nearest neighbors search," *Proc. VLDB Endow.*, vol. 18, no. 4, pp. 1118–1130, 2024, doi: 10.14778/3717755.3717770.

[31] J. Engels, B. Landrum, S. Yu, L. Dhulipala, and J. Shun, "Approximate nearest neighbor search with window filters," in *Proc. 41st Int. Conf. Mach. Learn. (ICML)*, ser. PMLR, vol. 235, 2024, pp. 12469–12490.

[32] M. Wang, L. Lv, X. Xu, Y. Wang, Q. Yue, and J. Ni, "An efficient and robust framework for approximate nearest neighbor search with attribute constraint," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 15738–15751, doi: 10.52202/075280-0692.

[33] Y. Cai, J. Shi, Y. Chen, and W. Zheng, "Navigating labels and vectors: A unified approach to filtered approximate nearest neighbor search," *Proc. ACM Manag. Data*, vol. 2, no. 6, pp. 1–27, 2024, doi: 10.1145/3698822.

[34] G. Gupta, J. Yi, B. Coleman, C. Luo, V. Lakshman, and A. Shrivastava, "CAPS: A practical partition index for filtered similarity search," arXiv:2308.15014, 2023.

[35] Z. Li, S. Huang, W. Ding, Y. Park, and J. Chen, "SIEVE: Effective filtered vector search with collection of indexes," *Proc. VLDB Endow.*, vol. 18, no. 11, pp. 4723–4736, 2025, doi: 10.14778/3749646.3749725.

[36] G. Sehgal and S. Salihoğlu, "NaviX: A native vector index design for graph DBMSs with robust predicate-agnostic search performance," *Proc. VLDB Endow.*, vol. 18, no. 11, pp. 4438–4450, 2025, doi: 10.14778/3749646.3749704.

[37] C. Ye, X. Yan, and E. Lo, "Compass: General filtered search across vector and structured data," arXiv:2510.27141, 2025.

[38] Microsoft DiskANN GitHub issue #550, "[Question] Low Recall in Filtered-Disk ANN on Sift Dataset-1M with Uniform and Zipf Distributions," https://github.com/microsoft/DiskANN/issues/550, accessed 2026.